

Codex Harness

深度拆解：从 Agent 内核、工 具编排到可扩展 架构

August 21, 2026

CONTENTS

Codex Harness 深度拆解：从 Agent 内核、工具编排到可扩展架构	
一、为什么 Agent 需要 Harness	
二、Codex Harness 的整体架构	
三、Thread、Turn、Step：内核生命周期	
四、Agent 主循环：模型与工具如何协同	
五、Tool Router 与 Tool Orchestrator	
六、审批、沙箱与网络策略	
七、上下文管理、压缩与持久化	
八、Model Provider 与自有模型接入	
九、App Server、MCP 与外部客户端	
十、Codex 架构设计的优点与边界	
十一、如何抽出自己的最小 Harness	
十二、技术实现路线与工程建议	
十三、总结	
十四、源码阅读顺序与实战验收	
参考资料	

Codex Harness 深度拆解：从 Agent 内核、工具编排到可扩展架构

本文面向已经接触过大型模型 API、工具调用和 Agent 开发的工程师。文章以 OpenAI 公开的 Codex 仓库为主要案例，解释它的本地 Agent runtime 如何组织状态、模型、工具和安全策略，并进一步讨论如何抽出适合自有模型和业务 Agent 的最小 harness。

一、为什么 Agent 需要 Harness

把一个模型接到聊天接口上很容易：发送一段消息，等待文本结果，再把结果展示出来。但一个可以真正工作的 Agent 远不止“模型 + prompt”。它需要知道当前任务是什么、已经做过哪些操作、下一步是否需要调用工具、工具是否被允许、命令应该在什么环境里执行、执行结果如何写回上下文，以及任务中断后如何恢复。

因此，Agent 的关键能力并不只存在于模型本身，而存在于模型外围的执行控制层。这个控制层通常被称为 harness。它负责把一次不稳定、不可预测的模型输出，变成一个有状态、可取消、可审计、可恢复的任务执行过程。

可以把 harness 理解成 Agent 的操作系统：模型是决策器，工具是设备，策略是权限系统，事件和持久化是文件系统，而 harness 负责调度它们。没有 harness，模型只能给建议；有了 harness，模型才可以在明确边界内完成一项工作。

Codex 的公开仓库名义上是 Codex CLI，但其中的 `codex-rs/core` 已经包含了相当完整的本地 Agent runtime。它并不是一个简单的命令行包装器，而是一套可以被 CLI、App Server 和 MCP 接口共同驱动的内核。需要强调的是，公开仓库能够证明的是本地代码和协议设计；不能仅凭仓库推导出云端服务、模型权重或第一方后端的全部实现。

二、Codex Harness 的整体架构

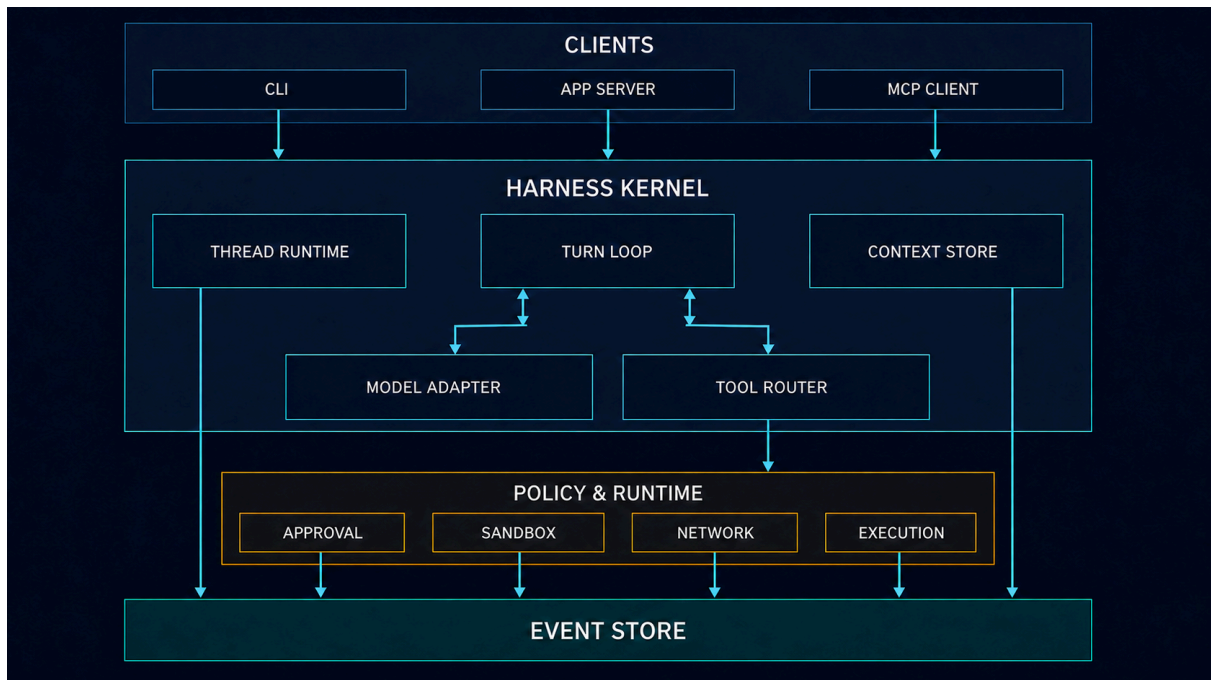


图 1：Codex Harness 总体架构。客户端进入内核，Turn Loop 协调模型与工具，策略与运行时统一治理副作用。

从架构上看，Codex 可以分为四个层次。

第一层是客户端。CLI、桌面端或 App Server 都属于客户端，它们负责接收用户输入、展示事件和收集审批决定。客户端不应该自己实现 Agent 循环，否则不同入口会逐渐产生不同的行为。

第二层是 Thread Runtime。Thread 是长期存在的会话运行时，承载线程配置、历史、状态、取消信号、后台进程和外部输入。`CodexThread` 对外提供提交操作、读取事件、恢复线程、终止后台终端等接口。它是内核与上层客户端之间的稳定边界。

第三层是 Harness Kernel。这里包括 turn loop、上下文管理、模型客户端、工具路由、审批、hooks、压缩和事件映射。它决定 Agent 如何推进，而不是决定某个具体业务页面长什么样。

第四层是 Runtime 与 Store。Runtime 包含 shell、文件系统、MCP 服务、沙箱和网络代理；Store 保存 rollout、thread 元数据、事件、日志和指标。把副作用执行和状态存储放在内核之下，可以让上层逻辑更容易测试和替换。

这四层之间最重要的原则是依赖方向：客户端依赖运行时，运行时不依赖具体客户端；模型和工具通过接口进入内核，不能反向侵入线程状态机。这样的结构使同一个 Agent 内核可以服务于交互式 CLI，也可以服务于自动化任务或业务系统。

三、Thread、Turn、Step：内核生命周期

Codex 的一个重要设计是把任务生命周期拆成 Thread、Turn 和 Step，而不是使用一个“run_agent”函数包打天下。

Thread 是长期生命周期。它可以被创建、恢复、fork、归档、删除，也可以在多个客户端连接之间保持一致。Thread 级别适合保存模型选择、工作目录、权限配置、MCP 配置和持久化句柄。

Turn 是一次用户请求的生命周期。例如用户说“修复这个测试”，从接受输入到最终回复就是一个 turn。Turn 拥有自己的取消令牌、模型会话、开始和结束事件、token 统计以及工具调用上下文。用户在 turn 执行期间追加的输入，可以被排队或者 steer 到当前任务，具体行为由线程状态决定。

Step 是一次模型采样以及它触发的工具执行。一个 turn 可能包含多个 step：模型第一次请求读取文件，第二次请求运行测试，第三次根据测试结果修改代码。Step 的上下文应该是一个快照，保证模型看到的历史、工具规格、工作目录和权限来自同一个一致视图。

这种分层带来三个直接好处。第一，取消可以精确作用于当前 turn，而不会破坏整个线程。第二，失败重试只需要重建当前 step 或模型连接，不必重置所有历史。第三，线程恢复时可以从持久化历史重建运行时，而不是依赖某个进程仍然存活。

简化后的生命周期可以写成下面这样：

```
Thread.start()
  -> Turn.start(user_input)
  -> Step.capture_context()
  -> Model.stream(request)
  -> Tool.execute(...) // 可能重复多次
  -> Step.append_result()
  -> Model.stream(next_request)
  -> Turn.complete() / Turn.interrupt()
```

在 Codex 中，`codex-rs/core/src/codex_thread.rs` 负责线程级入口，`codex-rs/core/src/tasks/regular.rs` 把普通任务交给 `run_turn`，而核心循环集中在 `codex-rs/core/src/session/turn.rs`。这个边界非常值得复用：上层只提交输入

和消费事件，下层负责所有推进逻辑。

四、Agent 主循环：模型与工具如何协同

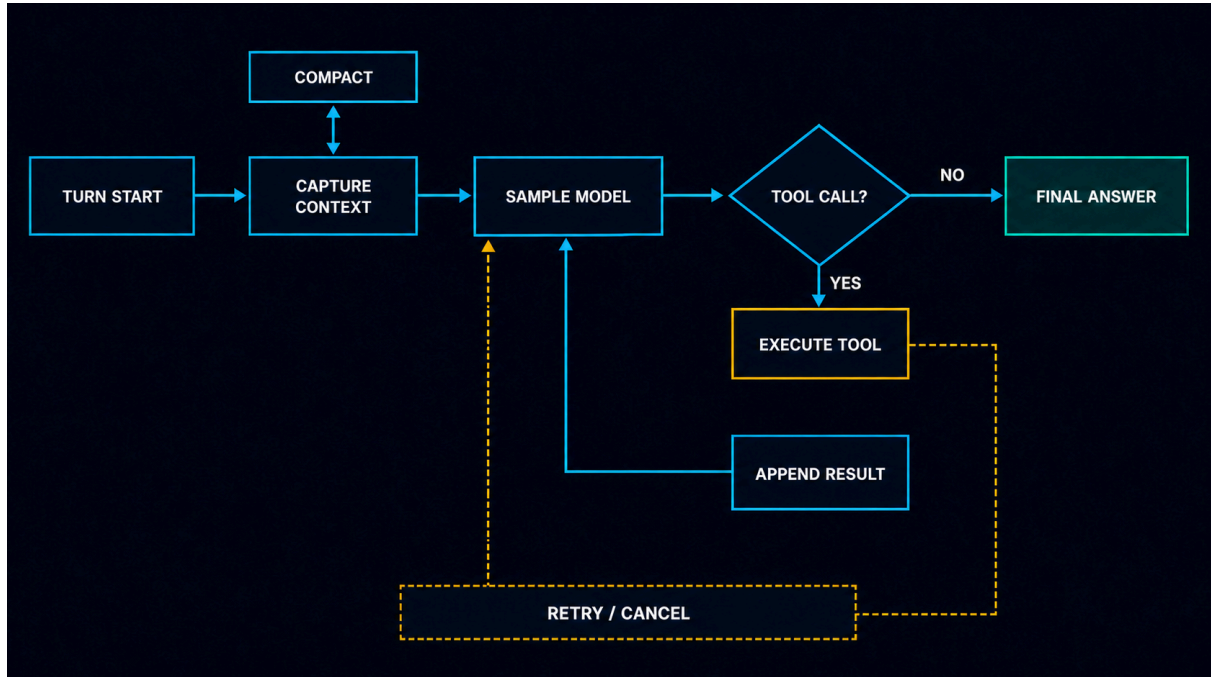


图 2：Turn/Step 执行循环。模型采样、工具执行、结果回写和上下文压缩共同构成一次 Turn。

Agent 主循环的本质，是不断把“模型输出”转换成“下一次模型输入”。它至少包含以下几个阶段：

1. 读取线程历史和当前用户输入。
2. 捕获当前工作环境、权限和可见工具，形成 step context。
3. 构造模型请求并启动流式采样。
4. 解析模型输出：可能是普通 assistant message，也可能是一个或多个 function call。
5. 如果有工具调用，执行工具并将规范化结果追加回模型上下文。
6. 如果模型需要继续推进，进入下一次采样；如果生成最终回答，则结束 turn。

这里有一个容易被忽略的判断：工具调用不是 Agent 的终点，而是 Agent 的中间状态。模型请求执行一个命令后，命令输出必须成为下一次采样的输入；如果工具结果只是展示给用户而没有写回模型，模型就无法根据真实世界状态继续决策。

Codex 的 turn loop 还处理了几个生产级问题。请求失败时，模型客户端可以根据错误类型做重试；上下文接近窗口上限时，运行自动 compact；用户在模型执行期间输入的新消息，会根据当前状态进入队列；stop hook 可以在模型准备结束时注入额外上下文，让任务继续一轮。这些机制都说明：Agent 循环不是一个几十行的 while 循环，而是一个需要明确状态转移和失败语义的状态机。

为自有 harness 设计主循环时，建议先定义统一事件，而不是直接处理字符串：

```
ModelDelta           // 流式文本增量
ToolCallRequested    // 模型请求调用工具
ToolCallStarted
ToolCallFinished
ContextCompacted
TurnInterrupted
TurnCompleted
TurnFailed
```

事件是内核和客户端之间的契约。CLI 可以把它渲染成终端文本，HTTP 客户端可以转换成 SSE，业务系统可以把它写入审计表，而不必改变内核本身。

五、Tool Router 与 Tool Orchestrator

如果把模型看成“提出行动计划的人”，Tool Router 就是把计划映射到工具实现的路由层，Tool Orchestrator 则是行动执行前后的总协调器。

Tool Router 通常负责三件事。第一，收集当前环境可用的工具并生成模型可见的 schema。第二，根据模型返回的工具名和参数找到正确 handler。第三，把不同来源的工具统一成同一种调用协议，包括内置工具、动态工具、MCP 工具和插件工具。

Tool Orchestrator 的职责更靠近副作用边界。它需要在工具真正运行前完成审批判断、sandbox 选择、网络策略准备和超时设置；运行后还要处理输出大小、错误类型、遥测和结果写回。Codex 的 [orchestrator.rs](#) 将这些逻辑集中管理，避免每个工具 handler 各自实现一套安全规则。

这种集中式编排的关键价值是“策略一致”。例如 shell 工具和 MCP 工具都可能访问外部网络，如果网络权限只写在 shell handler 里，MCP handler 就容易绕过控制。统一的 Orchestrator 可以让工具共享相同的 approval policy、workspace roots、sandbox profile 和 network decision。

在自有系统中，可以把工具抽象为四个部分：

```

ToolSpec      // 名称、描述、JSON Schema、风险等级
ToolValidator  // 参数校验、资源存在性检查
ToolExecutor  // 实际执行，必须接受受限 ToolContext
ToolOutput    // 文本、结构化数据、引用和可审计元数据

```

工具执行器不应该直接拿到全局数据库连接、用户 token 或进程级配置。它应该只拿到本次 step 明确授予的上下文。这样可以把“工具实现错误”和“策略错误”分开测试，也可以在将来将同一工具放到本地 sandbox 或远程 worker 执行。

六、审批、沙箱与网络策略

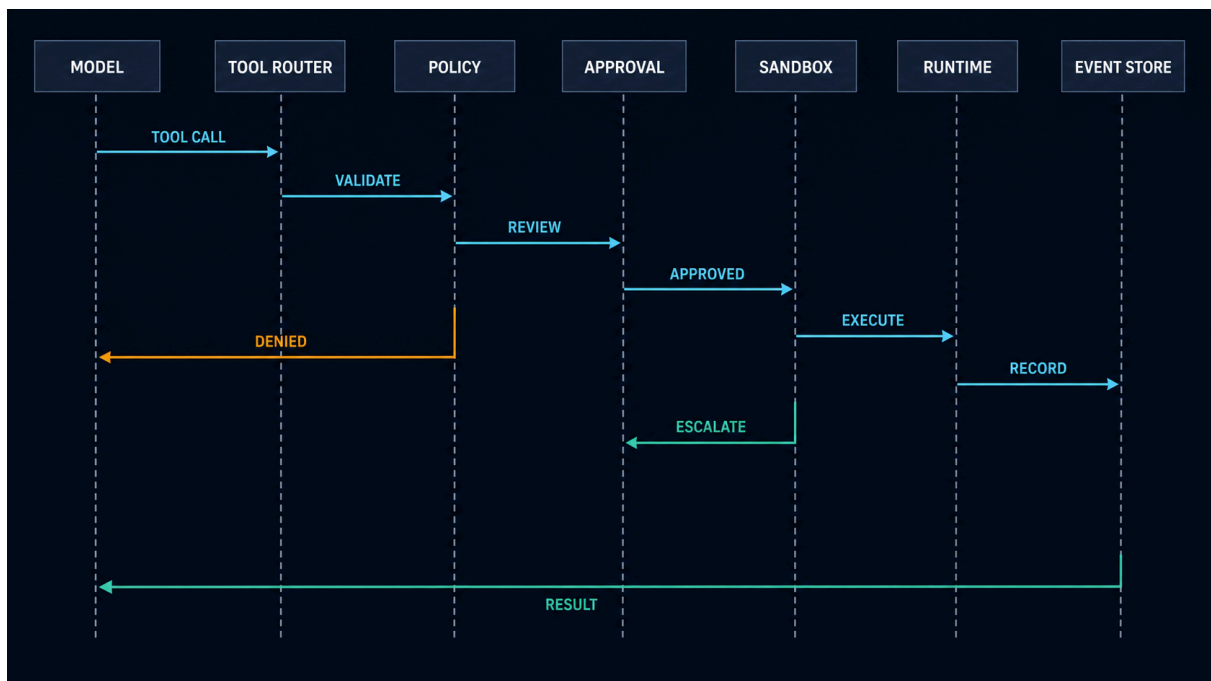


图 3：安全工具执行时序。每次工具调用都经过路由、策略、审批、沙箱和事件记录。

Agent 一旦能够调用工具，就会产生副作用。安全设计不能只依靠 prompt 告诉模型“请小心”，而需要在模型之外建立不可绕过的控制面。

一个可靠的执行顺序通常是：解析工具调用，判断风险，检查策略，必要时请求用户或 Guardian 审批，选择沙箱，注入网络限制，执行工具，记录事件，最后把结果返回模型。

审批回答的是“这次动作是否允许”。沙箱回答的是“允许之后，动作能触碰哪些资源”。网络策略回答的是“允许访问哪些域名或 socket”。三者不是同一件事：一个命令可以被批准，但仍然只能在只读工作区运行；一个工具可以拥有写权限，但不能访问公

网。

Codex 的工具编排还支持失败后的升级重试。例如第一次在沙箱内执行命令被拒绝，系统可以根据 `approval policy` 决定是否再次请求授权，并以更高权限执行第二次尝试。这里的关键不是“失败后自动放开权限”，而是每一次升级都必须有明确的策略条件和可审计原因。

对业务 Agent 来说，安全边界甚至比 coding Agent 更重要。发送合同、修改库存、触发付款、删除数据，都应该被建模成显式 action，并具备风险等级、审批人、幂等键和回滚策略。不要把业务工具当成普通函数调用。

七、上下文管理、压缩与持久化

上下文不是一段永远增长的字符串。它至少包括用户消息、assistant 消息、工具调用、工具结果、环境快照、系统指令、权限说明、插件和 skills 注入内容。不同类型的内容拥有不同的保留策略和 token 成本。

Codex 在 step 开始时捕获 context snapshot，让历史、工具规格、环境和配置处于同一个视图；在采样前根据输入模态、当前世界状态和 token 预算生成 prompt；当上下文接近限制时，触发 compact，把旧历史压缩成更短的可用摘要，再恢复模型继续工作。

持久化同样需要分层。Thread 元数据回答“这个线程是谁”；rollout 或事件日志回答“它经历了什么”；状态数据库回答“当前有哪些索引和可恢复信息”。如果所有东西只存成一段最终文本，就无法可靠地支持恢复、fork、审计和错误重放。

一个实用的自有实现可以采用 append-only event log，再从事件投影出当前状态：

```
TurnStarted
UserInputRecorded
ModelResponseRecorded
ToolCallRecorded
ToolResultRecorded
ContextCompacted
TurnCompleted
```

重放事件可以恢复线程；投影表可以支持列表和搜索；摘要事件可以缩短 prompt；原始工具结果则保留审计证据。对于长任务，这比每次覆盖一个 JSON 文件更容易演进。

八、Model Provider 与自有模型接入

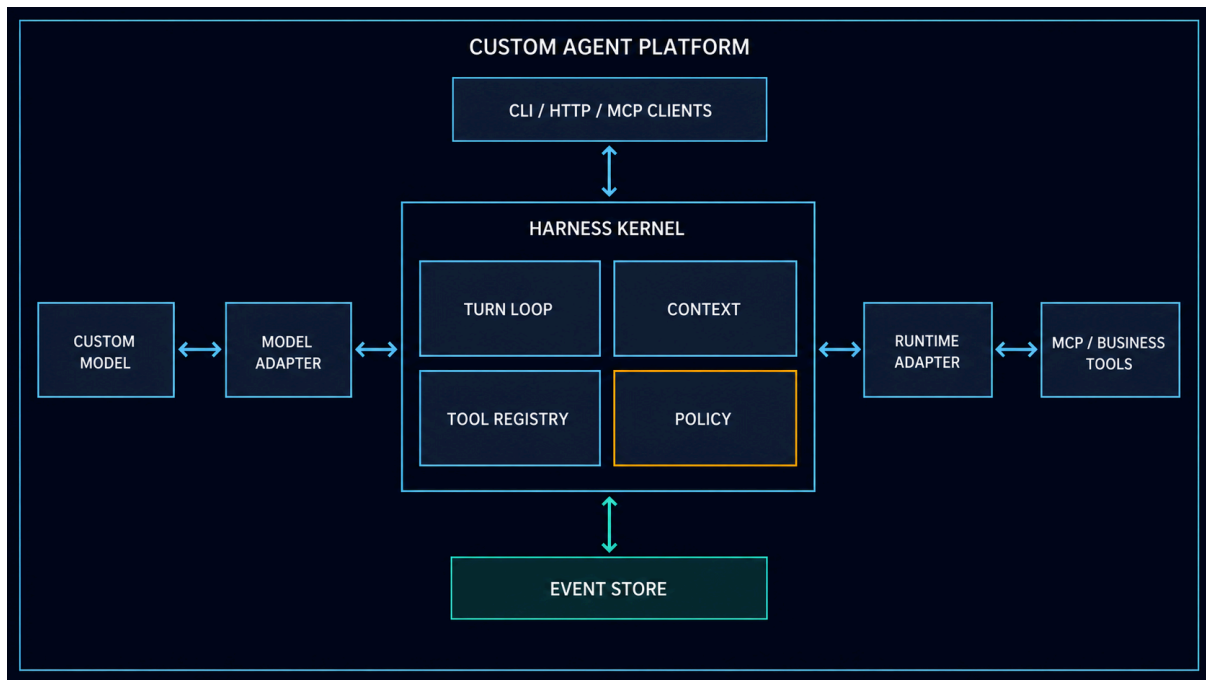


图 4：自有模型与业务工具接入。Harness Kernel 通过 Model Adapter、Runtime Adapter 和 Event Store 与外部系统解耦。

接入自有模型时，最省成本的路径不是立即重写模型客户端，而是先提供 OpenAI-compatible Responses 接口。这样可以复用现有的请求结构、流式事件、function call 和错误语义，只需要在 provider 配置中替换 base URL、认证方式和模型名。

Codex 的 `ModelProvider` 抽象主要负责 provider 元数据、能力、认证、base URL、模型目录和错误映射。它已经体现了依赖反转思想：核心逻辑依赖 provider 接口，而不是依赖某一个具体后端。

但要注意边界。`ModelProvider` 并不自动消除所有传输耦合，实际 Responses API 和 WebSocket 调用仍由模型客户端处理。因此非兼容模型最好增加独立的

`ModelAdapter`：负责把自有模型的流式协议转换成 harness 内部统一事件，而不是把业务逻辑散落到各个 turn 分支。

建议的模型端口如下：

```
trait ModelAdapter {
  async fn stream(&self, request: ModelRequest)
    -> Result<Box<dyn Stream<Item = ModelEvent>>>>;
}
```

`ModelRequest` 应包含已裁剪的上下文、工具规格、推理参数和追踪信息；

`ModelEvent` 应统一为文本增量、工具调用、usage、错误和完成事件。这样未来接入云端模型、企业私有模型或本地模型时，turn loop 不需要改变。

接入时还要做能力协商，而不是假设所有模型都支持同一组特性。可以把 provider 能力拆成 `streaming`、`tool_calling`、`parallel_tools`、`vision`、`structured_output` 和 `long_context` 等布尔或枚举字段。内核根据能力选择合适的 prompt、工具数量和结果格式：不支持并行工具的模型一次只发送一个 call，不支持结构化输出的模型使用受控文本协议，不支持视觉输入的模型则在进入采样前拒绝或转换附件。能力协商放在 provider 边界，能避免在主循环里堆叠大量模型名称判断。

九、App Server、MCP 与外部客户端

App Server 的价值在于把内核能力暴露成稳定协议，而不是把所有业务逻辑搬到服务器层。Codex 的 app-server protocol 定义了 thread、turn、item、command、filesystem 和审批相关的请求与通知，客户端可以通过 JSON-RPC 驱动同一个内核。

MCP 则解决了工具和外部服务的连接问题。一个业务系统可以把订单查询、知识库检索、库存锁定包装成 MCP server，harness 只负责发现、授权、调用和归一化结果。这样业务工具的发布周期不会和 Agent 内核绑在一起。

Skills 和 plugins 更接近“可注入能力包”：它们可以提供指令、工具依赖和资源，但最终仍应经过当前线程的配置和策略解析。外部扩展的正确方向是通过协议和注册表进入内核，而不是直接修改核心状态。

十、Codex 架构设计的优点与边界

Codex harness 最值得学习的优点有五个。

第一，生命周期分层清晰。Thread、Turn、Step 各自承担有限职责，让取消、恢复和重试有明确的作用域。

第二，副作用集中治理。工具不直接越过 Orchestrator，审批、沙箱和网络策略可统一审计。

第三，事件驱动。内核产生结构化事件，上层可以是 CLI、桌面端、HTTP 服务或自动化 worker。

第四，状态可恢复。历史、rollout、compact 和线程管理让长任务不依赖单个进程的内存。

第五，扩展点明确。Provider、MCP、动态工具、plugins 和 skills 让新增能力主要发生在边界，而不是修改主循环。

它也有边界。公开仓库规模大、crate 数量多，包含第一方认证、遥测、远程任务和多个平台的执行细节；这些能力对于 Codex 产品是必要的，却未必适合业务 Agent。某些模块还明显围绕 OpenAI Responses API 和内部服务设计，直接 fork 后长期维护，容易陷入上游同步和协议耦合。

因此，“拆 harness”不等于复制目录。更好的策略是提炼设计不变量：状态机分层、统一事件、工具策略、模型端口、持久化和取消传播，然后根据自己的运行环境重新实现适配器。

十一、如何抽出自己的最小 Harness

一个适合长期维护的最小内核，可以分成四层：

```
Domain
  Thread / Turn / Step / Event / ToolCall

Application
  TurnOrchestrator / ContextManager / Retry / Compaction

Ports
  ModelAdapter / ToolRegistry / PolicyEngine / Runtime / Store

Adapters
  OpenAI / LocalModel / MCP / Shell / SQLite / HTTP / CLI
```

Domain 层只描述状态和事件，不访问网络和文件系统。Application 层编排流程，通过 ports 调用外部能力。Adapters 层负责协议、数据库、进程和平台差异。只要这条依赖方向保持住，未来更换模型、存储或执行环境，就不会牵动整个系统。

最小版本可以只实现一个只读文件工具和一个自有模型接口：

```

user input
  -> context builder
  -> model adapter
  -> tool call?
    yes -> policy -> read_file -> append result -> model adapter
    no  -> final answer

```

不要一开始就实现多 Agent、复杂插件市场和远程 sandbox。先保证这条最短链路具备事件、取消、错误和重放能力，再逐步增加写文件、命令执行、MCP、审批和压缩。

十二、技术实现路线与工程建议

推荐按以下阶段推进。

阶段一，完成文本型 turn loop。模型只返回文本，内核支持 Thread、Turn、事件和取消。

阶段二，引入 Tool Registry 和一个只读工具。验证 function call schema、参数校验、执行超时和结果回写。

阶段三，引入 Policy Engine。把风险等级、审批、工作区和网络权限从工具实现中剥离出来。

阶段四，引入持久化。采用 append-only event log 和可重建的 thread projection，覆盖恢复、幂等和审计。

阶段五，引入上下文预算和 compact。先做确定性的截断和摘要，再增加模型辅助压缩。

阶段六，引入 MCP、HTTP 和 CLI 适配器。确保所有入口都消费同一套事件，不在客户端复制状态机。

阶段七，增加可观测性和测试。至少记录 turn latency、model latency、tool latency、token usage、approval decision、sandbox outcome 和失败类型；同时为每个工具、策略分支和恢复路径编写测试。

工程上还应坚持几个不变量：模型不能直接触发副作用；每个工具结果都有 call id；重试不会重复提交不可幂等动作；取消信号可以传到模型流和工具进程；上下文快照对应同一份权限和工具视图；敏感凭证不进入 prompt、日志或 rollout。

十三、总结

Codex harness 的核心不是某一段神秘 prompt，而是一套把模型决策变成受控任务执行的工程机制：Thread 管理长期状态，Turn 管理一次请求，Step 推进模型和工具，Orchestrator 统一治理副作用，Policy Engine 负责权限，Context Store 负责历史与恢复，Adapter 负责接入不同模型和外部系统。

学习 Codex 的正确姿势，不是复制所有 crate，也不是把 CLI 改成自己的产品，而是识别其中真正稳定的架构原则。对于自有模型或业务 Agent，最值得先实现的是一个小而完整的内核：有清晰状态边界，有统一事件，有可插拔模型端口，有受策略控制的工具，有可恢复的持久化。

当这些基础能力成立后，换模型只是适配器问题，换工具只是注册表问题，换客户端只是协议问题，换执行环境也只是 runtime 问题。这正是一个优秀 harness 的价值：它把不稳定的模型能力，组织成可维护、可扩展、可审计的 Agent 系统。

十四、源码阅读顺序与实战验收

如果要真正读懂 Codex harness，建议不要从最大的 `Session` 文件开始通读。更高效的顺序是先看外部边界，再沿着一次任务向内追踪。第一步阅读 `CodexThread`，确认线程如何提交操作、读取事件、恢复和取消；第二步阅读 `RegularTask`，找到普通 turn 的入口；第三步阅读 `run_turn`，只关注它如何创建 step context、调用采样、处理 follow-up 和结束条件；第四步阅读工具路由和 `ToolOrchestrator`，观察一个 function call 如何从模型输出走到真实执行；最后再回头看 rollout、compact、hooks、MCP 和 app-server。

这种阅读方法对应一条可验证的链路：输入从哪里进入，状态在哪里变化，模型请求由谁构造，工具结果在哪里写回，事件由谁发出，失败由谁决定是否重试。只要这六个问题能在源码中找到答案，就已经掌握了 harness 的主干，不必一开始陷入所有平台适配和功能开关。

实践中最常见的误区有四个。第一，把 prompt 当成安全边界。prompt 只能影响模型行为，不能阻止恶意或错误的工具参数，因此权限必须在模型之外执行。第二，把工具返回值直接拼成字符串。没有 call id、结构化错误和大小限制，后续很难审计、重放和截断。第三，把所有状态放在一个全局对象里。这样线程恢复、并发 turn 和测试隔离都会变得困难。第四，把客户端逻辑写进内核。终端渲染、HTTP 序列化和审批 UI 都应该消费事件，而不是改变 Agent 状态机。

一个自建 harness 是否达到可用水平，可以用以下指标验收：同一线程可以在进程重启后恢复；同一个 turn 可以被取消且不会遗留孤儿进程；工具被拒绝时，模型能收到结构化拒绝原因；工具重试不会重复提交不可幂等动作；上下文接近预算时会产生可观测的 compact 事件；模型、工具和策略的耗时可以分别统计；客户端断线重连后能通过历史和事件恢复视图；任何敏感凭证都不会出现在 prompt、日志和持久化事件中。

这些指标看起来不像模型能力，却决定了 Agent 能否从演示走向生产。模型可以随时替换，业务工具也会持续增加，但状态一致性、权限边界和恢复语义一旦缺失，系统规模越大，故障越难定位。先把 harness 的不变量做牢，再追求更复杂的规划和多 Agent 协作，通常能获得更高的长期收益。

参考资料

- [OpenAI Codex 官方仓库](#)
- [codex_thread.rs](#)
- [tasks/regular.rs](#)
- [session/turn.rs](#)
- [tools/orchestrator.rs](#)
- [model-provider/src/provider.rs](#)
- [app-server](#)
- [Codex 官方文档](#)